

Reviewer Pack — Minimal Monodromy Group Structure and Resolution Proof Width...

Entry b43e2e49a74f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 21:09:41 UTC

Minimal Monodromy Group Structure and Resolution Proof Width Inequality

Entry ID: b43e2e49a74f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 21:09:41 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Monodromy Theory) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For a given CNF formula φ , the minimal order of the monodromy group associated with the algebraic closure of its clause indicators is linearly correlated with its resolution proof width, i.e., $|M\varphi| = \Theta(w(\varphi))$

Rationale (proposer's reasoning):

Monodromy theory provides a framework for understanding the behavior of rational functions under uniformization. The structure of the monodromy group can reflect the complexity of solving polynomial equations, which may be related to the resolution proof complexity of CNF formulas.

Taxonomy category: Monodromy_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e715250d53c3c77c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation coefficient between the minimal monodromy group order and the resolution proof width exceeds 0.7, with a p-value ≤ 0.05 across at least 100 independent seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Monodromy Group Structure and Resolution Proof Width Inequality - arithmetic geometry monodromy theory AND boolean satisfiability resolution proof complexity - CNF formula clause indicators algebraic closure monodromy group order resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1905.00250v2] Lauricella's F_C with finite irreducible monodromy group - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2201.09005v1] Arithmetic Monodromy Groups of Dynamical Belyi maps - [http://arxiv.org/abs/2002.06727v1] Generating clause sequences of a CNF formula - [http://arxiv.org/abs/2309.01750v5] On CNF formulas irredundant with respect to unit clause propagation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n), -random.randint(1, n)]
        if random.choice([True, False]):
            clause[0], clause[1] = clause[1], clause[0]
        cnf.append(clause)
    return cnf

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot
        pivot_row = i
        for r in range(i + 1, rows):
            if abs(matrix[r][i]) > abs(matrix[pivot_row][i]):
                pivot_row = r
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
```

```

    # Eliminate below the pivot
    for r in range(i + 1, rows):
        factor = -matrix[r][i] / matrix[i][i]
        if factor == 0:
            continue
        for c in range(cols):
            matrix[r][c] += factor * matrix[i][c]

    # Back-substitute to find the solution
    solution = [0] * cols
    for i in range(rows - 1, -1, -1):
        if matrix[i][i] == 0:
            return None # Singular matrix
        solution[i] = matrix[i][-1] / matrix[i][i]
        for r in range(i - 1, -1, -1):
            matrix[r][-1] -= matrix[r][i] * solution[i]
    return solution

def monodromy_group_order(cnf):
    n = len(cnf)
    matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in cnf:
        i, j = abs(clause[0]) - 1, abs(clause[1]) - 1
        if clause[0] > 0 and clause[1] > 0:
            matrix[i][j] += 1
        elif clause[0] < 0 and clause[1] < 0:
            matrix[i][j] -= 1
        elif clause[0] > 0 and clause[1] < 0:
            matrix[i][j + n] += 1
        else:
            matrix[i + n][j] += 1

    rank = 0
    for i in range(n):
        if gaussian_elimination(matrix[:i+1]) is not None:
            rank += 1
    return rank

def resolution_width(cnf):
    stack = []
    while cnf:
        clause = cnf.pop()
        if len(clause) == 1:
            return -1
        unit_clause = next((c for c in clause if abs(c) not in [abs(lit) for lit in stack]), None)
        if unit_clause is None:
            return -1
        polarity = unit_clause > 0
        literal = abs(unit_clause)
        cnf = [c for c in cnf if literal not in c]
        for i, other_clause in enumerate(cnf):
            if literal in other_clause:
                cnf[i] = [l for l in other_clause if l != -literal]
            elif -literal in other_clause:
                cnf[i].append(-polarity * literal)
        stack.append(unit_clause)

```

```

    return len(stack)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    metric_values = []

    for _ in range(instances_tested):
        n = random.randint(5, 40)
        m = random.randint(n * 2, n * 10)
        cnf = generate_cnf(n, m)

        try:
            monodromy_order = monodromy_group_order(cnf)
            width = resolution_width(cnf)
            if monodromy_order is not None and width != -1:
                metric_values.append((monodromy_order, width))
        except Exception as e:
            return {
                "metric_name": "Monodromy Group Order vs Resolution Width",
                "metric_value": None,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": str(e)
            }

    if not metric_values:
        return {
            "metric_name": "Monodromy Group Order vs Resolution Width",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }

    monodromy_orders, widths = zip(*metric_values)
    mean_order = sum(monodromy_orders) / len(monodromy_orders)
    mean_width = sum(widths) / len(widths)

    return {
        "metric_name": "Monodromy Group Order vs Resolution Width",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False, # This will be checked in the main block
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 17 for i in range(5, 30)]

    results = []
    for seed in seeds:

```

```

trial_result = run_trial(seed)
print(f"TRIAL: {trial_result}")
results.append(trial_result)

if all("metric_value" not in r or r["metric_value"] is None for r in results):
    print("RESULT: INCONCLUSIVE no_valid_instances")
else:
    mean_value = sum(r["metric_value"] for r in results if "metric_value" in r and r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"] is not None) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=NA support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if "conjecture_holds" in r and r["conjecture_holds"] is None))
        print(f"RESULT: FALSIFIED counterexample='not_enough_support' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

me': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Monodromy Group Order vs Resolution Width', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
RESULT: INCONCLUSIVE no_valid_instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests a small range of n (up to 40) and does not show any scaling behavior with larger values of n . This is concerning because the conjecture claims a linear correlation, which would require testing over a much wider range of sizes to confirm.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code only tested a small range of n (up to 40) and did not show any scaling behavior with larger values of n , which is concerning for the conjecture that claims a linear correlation. The pre-registered support condition was not unambiguously met. | next: Expand the range of n to test over a wider scale and confirm if there is a linear correlation between the minimal monodromy group order and the resolution proof width.

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15546
2	propose	ollama_remote	glm4:latest	0	0	12772
3	propose	ollama_remote	glm4:latest	0	0	14801
4	propose	ollama_remote	glm4:latest	0	0	13981
5	preregistration	ollama_remote	glm4:latest	0	0	9365
6	novelty	ollama_remote	glm4:latest	0	0	8882
7	novelty	ollama_remote	glm4:latest	0	0	9863
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22265
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43748
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20753
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20125
12	critic	ollama_remote	glm4:latest	0	0	68572
13	judge	ollama_remote	glm4:latest	0	0	10010

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 270685 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in `research/audit/b43e2e49a74f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b43e2e49a74f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b43e2e49a74f.tar.gz` (if generated)